Secure AI Systems: Red and Blue Teaming an MNIST Classifier

Oct 24, 2026

Team members: (Satyasheel Singh, cs26resch02001) · (Priya Agrawal, cs26mtech02008)

We built a small convolutional neural network in PyTorch that classifies the clean MNIST test set with 99.09% accuracy, and then attacked and defended it. A white-box PGD attack at ε = 0.3 brought its accuracy down to 0.00%. Poisoning 5% of the training data planted a backdoor that fired on 99.94% of triggered digits, while clean accuracy stayed the same. PGD adversarial training brought accuracy under the same attack back up to 90.34% (83.30% against the strongest attack we tried), for a cost of 0.55 points of clean accuracy. Activation clustering found 95.4% of the poisoned images, and after retraining without them the backdoor worked on only 0.18% of triggered digits.

All code, models, datasets and results are in our repository, github.com/ItseXec/mnist-secure-ai. Every figure in this report is taken from files committed there, mainly in `results/` and `scans/`. We produced them on one machine: Ubuntu, Python 3.12.3, PyTorch 2.14.1 (CPU only, 4 threads) and the Adversarial Robustness Toolbox (ART) 1.20.1. All random seeds are fixed, so the commands in the README reproduce the same numbers.

1. CNN implementation for MNIST (GitHub repository)

Our classifier is a small two-block convolutional network with 225,034 trainable parameters. We wrote it in PyTorch and trained it for 5 epochs, which took 74.5 seconds on a laptop CPU.

Repository: https://github.com/ItseXec/mnist-secure-ai

Part	File
Model definition	`src/model.py`
Data download, integrity checks, train/validation split	`src/data.py`
Training	`src/train.py`
Evaluation (metrics, confusion matrix, inference time)	`src/evaluate.py`

Part	File
Trained model (weights only) and its SHA-256	`models/baseline.pt`, SHA-256 `ee46209e…bba28`
Baseline results	`results/baseline/`

We normalise the input inside the model itself, so images enter in the raw 0–1 pixel range. This keeps every attack strength (ε) in this report in pixel units. The layers are:

Layer	Output shape	Parameters
Normalise (MNIST mean 0.1307, std 0.3081)	1 × 28 × 28	0
Conv 3×3, 32 filters + ReLU	32 × 26 × 26	320
Max-pool 2×2	32 × 13 × 13	0
Conv 3×3, 64 filters + ReLU	64 × 11 × 11	18,496
Max-pool 2×2, flatten	1,600	0
Fully connected + ReLU + dropout 0.25	128	204,928
Fully connected (output)	10	1,290

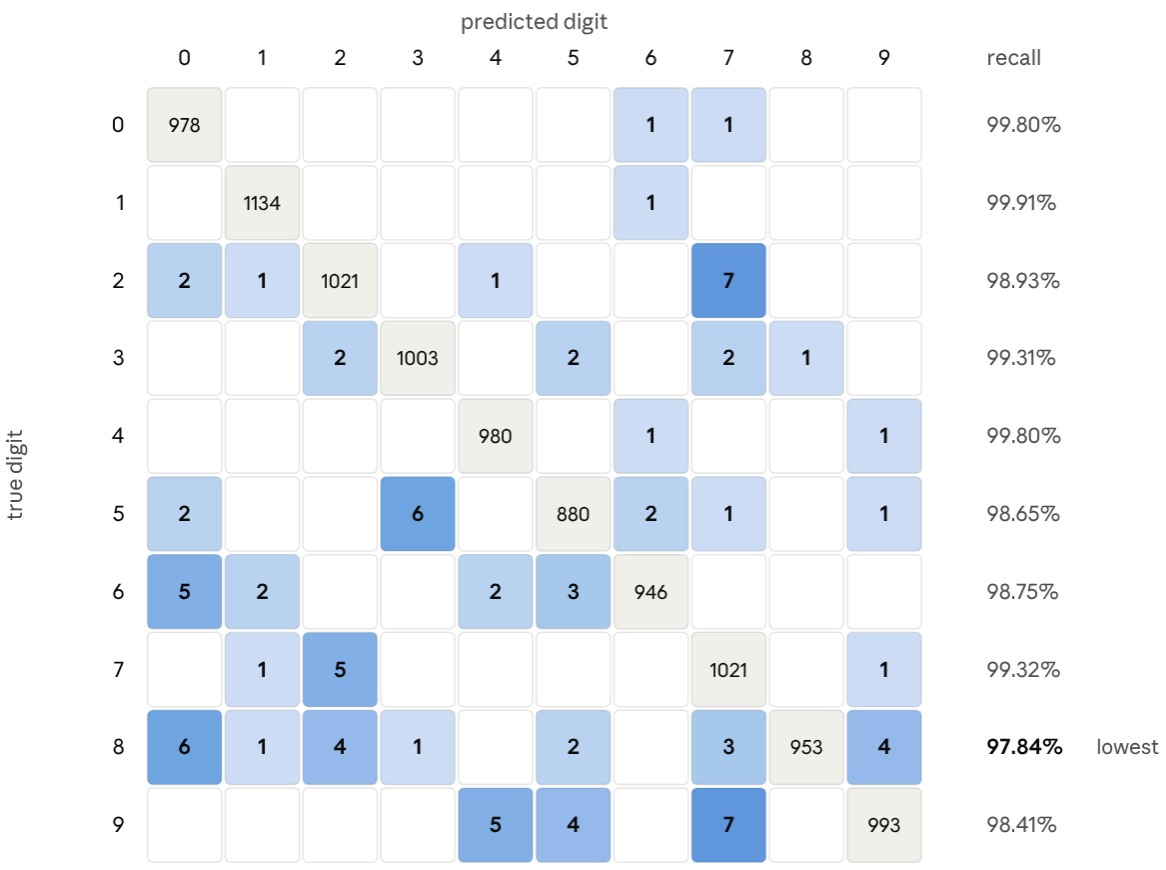
We split the 60,000 MNIST training images into 55,000 for training and 5,000 for validation (seed 42), and kept the 10,000-image test set for evaluation only. Training used the Adam optimiser with a learning rate of 0.001, batches of 128 and cross-entropy loss. After the fifth epoch the model reached 99.00% on the validation set. The MNIST files are downloaded over HTTPS and checked against pinned SHA-256 hashes before use; this is one of the SAST fixes described in Section 4.

2. Performance metrics of classification

On the clean test set of 10,000 images the baseline got 9,909 right (99.09%), with a cross-entropy loss of 0.0297. A single prediction takes about 0.23 ms on the CPU.

Metric	Value
Accuracy	99.09% (9,909 of 10,000)
Loss (cross-entropy)	0.0297
Macro precision / recall / F1	99.09% / 99.07% / 99.08%

Metric	Value
Inference time, one image at a time	0.229 ms median, 0.330 ms 95th percentile
Throughput, batches of 256	9,344 images per second



results/baseline/metrics.json · baseline model on the 10,000-image MNIST test set

Most of the 91 mistakes are between digits that look alike. Digit 8 has the lowest recall at 97.84%, and six of its 21 errors were read as 0. Digit 7 has the lowest precision (97.98%) because some 2s and 9s were predicted as 7, while digit 1 was almost never missed (99.91% recall).

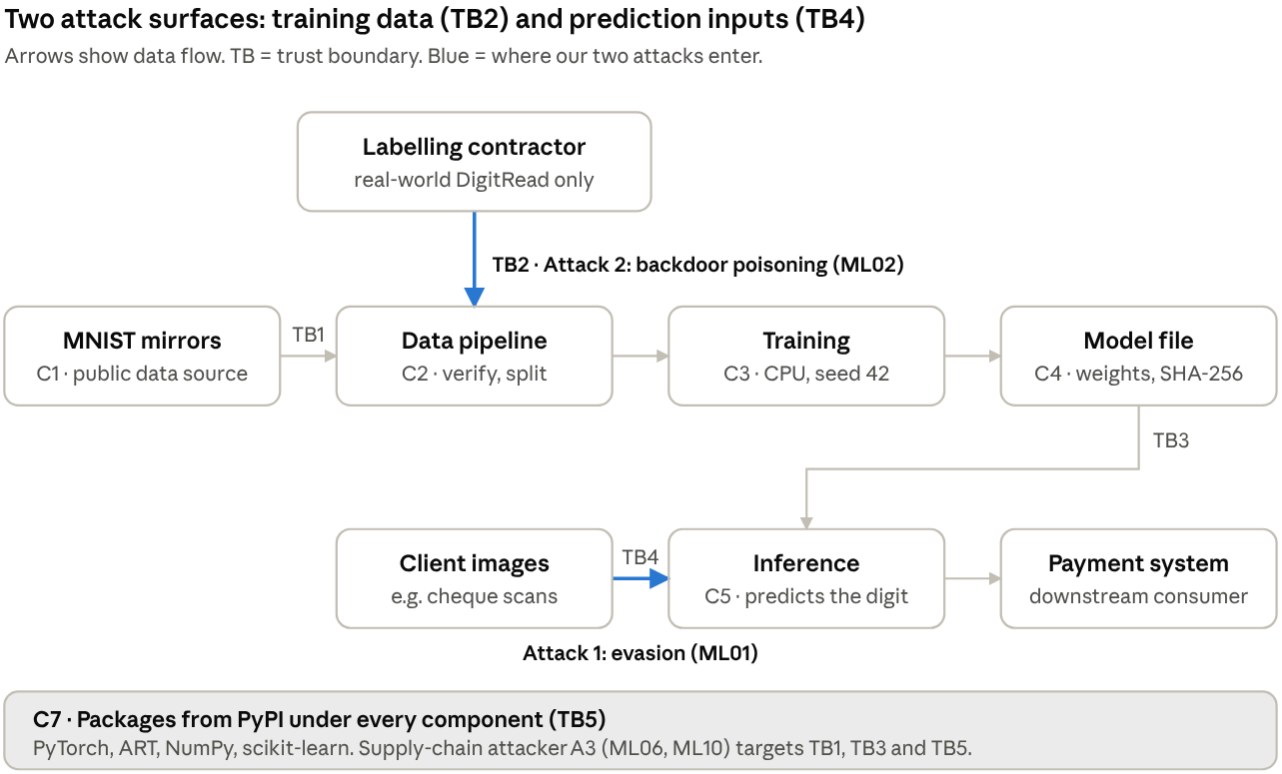
For inference time we report the median of 500 single-image predictions after a short warm-up, using 4 CPU threads. The same model measured anywhere from 0.229 to 0.37 ms depending on what else the laptop was doing. The final figures therefore come from `scripts/timing.py`, which times all five models in this report together over five interleaved rounds.

3. Threat model

We modelled the system as a data flow and asked where an attacker could get in. Two entry points come with the design itself: the images sent for prediction and the data used for training. We attacked both (Section 5) and defended both (Section 7). The remaining threats concern the files the pipeline loads, such as model files, datasets and packages, and four of these were fixed through the SAST work in Section 4. The full threat model is in `docs/THREAT_MODEL.md`.

3.1 System, real-world version and trust boundaries

MNIST on its own is a toy problem. To make the threats concrete, we imagined our classifier as the core of a service we call DigitRead, which a bank could use to read the handwritten amount on scanned cheques. DigitRead would be retrained from time to time on new samples labelled by an outside contractor. Compared with our lab setup, it adds a network API, real customers' handwriting and an external data supplier.



data-flow diagram · components C1–C7, trust boundaries TB1–TB5

The diagram shows two routes into the system that do not require touching any code: the training data (TB2) and the images sent for prediction (TB4). The model file (TB3) and the downloaded data and packages (TB1, TB5) are supply-chain routes, which we addressed through SAST.

Boundary	What crosses it	Why it matters
TB1 Internet → data pipeline	Raw training and test data	A spoofed or compromised mirror could deliver altered data
TB2 Contributors → training set	New labelled samples	Whoever supplies data can poison it (Attack 2)
TB3 Storage → inference	The model file	Loading a pickled model file can execute code
TB4 Client → inference	Images to classify	Every input is attacker-controlled (Attack 1)
TB5 Package registry → environment	Libraries (PyTorch, ART, NumPy, scikit-learn)	A compromised package runs with full privileges

3.2 Attacker model

	A1: Evasion attacker	A2: Poisoning attacker	A3: Supply-chain attacker
Exercised by	Attack 1 (ML01)	Attack 2 (ML02)	SAST findings (ML06, ML10)
Goal	Any wrong prediction for a chosen input (untargeted)	A hidden backdoor: trigger → digit 9, while clean accuracy stays normal	Run code on, or alter the model in, the training or inference host
Knowledge	White-box: architecture and weights (worst case, also covers a leaked model). Black-box transfer tested separately	None of the model or code; knows the task and the labels	The file formats the pipeline loads
Capability	Changes each pixel by at most ε (L_∞ norm) at prediction time	Controls 5% of the training set (2,750 images)	Can replace one model, dataset or mirror file
Entry point	TB4	TB2	TB1, TB3, TB5
Cost	1 (FGSM) to 20 (PGD) gradient computations per image	Labelling a few thousand images	Getting one file into the pipeline

We left two attackers out of scope: an insider with write access to the source code, and anyone with physical access to the hardware.

3.3 STRIDE, interpreted for machine learning and mapped to the OWASP ML Top 10

ID	STRIDE	Threat (ML interpretation)	OWASP ML	Mitigation	Status
T1	Spoofing	A fake or compromised mirror serves altered MNIST files	ML02, ML06	HTTPS-only download, SHA-256 pinned per file	Fixed
T2	Spoofing	A substituted model file is loaded as the genuine one	ML10	SHA-256 verified before loading	Fixed

ID	STRIDE	Threat (ML interpretation)	OWASP ML	Mitigation	Status
T3	Tampering	Adversarial inputs: small pixel changes cause wrong predictions	ML01	PGD adversarial training	Attacked and mitigated
T4	Tampering	Backdoor poisoning of the training data	ML02	Activation clustering, then retraining	Attacked and mitigated
T5	Tampering	Model weights altered in storage	ML10	Checksum on load; signing recommended	Partly fixed
T6	Tampering	Predictions altered on the way to the payment system	ML09	Integrity-protected transport, signed responses	Recommended
T7	Tampering	Gradual drift from manipulated feedback used in retraining	ML08	Review and rate-limit feedback; monitor drift	Recommended
T8	Repudiation	No record of who supplied which samples, so poison cannot be traced	ML02	Per-sample provenance; data and model hashes recorded per run	Partly done (model hash recorded)
T9	Information disclosure	Model inversion reconstructs training samples	ML03	Labels-only output; differential privacy (DP-SGD)	Recommended
T10	Information disclosure	Membership inference reveals whether a person's sample was used	ML04	DP-SGD; limit output precision	Recommended
T11	Information disclosure	Model theft through repeated API queries	ML05	Authentication, rate limits, labels-only output	Recommended
T12	Denial of service	Oversized or endless download fills the disk or hangs the job	ML06	30 s timeout, 20 MB size limit	Fixed
T13	Denial of service	Request flooding of the prediction API	—	Rate limits, input size checks	Recommended
T14	Elevation of privilege	Pickled model file runs code when `torch.load` opens it	ML06, ML10	Weights-only saving, `weights_only=True`, PyTorch ≥ 2.10	Demonstrated, then fixed
T15	Elevation of privilege	Dataset file with pickled objects runs code on `np.load`	ML06	`allow_pickle=False`, content checks	Fixed
T16	Elevation of privilege	Compromised or vulnerable dependency	ML06	Pinned versions, pip-audit, minimum safe versions	Partly fixed (hash locking recommended)

T14 is more than a theoretical risk. One of our tests in `tests/test_security.py` creates a model file whose hidden payload runs as soon as the version 1 loader opens it. It then shows that the fixed loader refuses the same file.

3.4 OWASP Machine Learning Security Top 10 coverage

Risk	Applies?	Treatment in this project
ML01 Input Manipulation	Yes: every input crosses TB4	**Attack 1** (FGSM, PGD); **Defense 1** (adversarial training)
ML02 Data Poisoning	Yes: training data crosses TB1 and TB2	**Attack 2** (backdoor); **Defense 2** (activation clustering); SHA-256 pinning at TB1
ML03 Model Inversion	Real-world version	Discussed (T9, LINDDUN)
ML04 Membership Inference	Real-world version	Discussed (T10, LINDDUN)
ML05 Model Theft	Real-world version	Discussed (T11)
ML06 AI Supply Chain	Yes: downloaded data, model files, PyPI packages	SAST fixes (T1, T12, T14, T15) and dependency audit
ML07 Transfer Learning	No: trained from scratch, no pre-trained weights	Not applicable
ML08 Model Skewing	Real-world version	Discussed (T7)
ML09 Output Integrity	Real-world version	Discussed (T6)
ML10 Model Poisoning	Yes: the weights file can be replaced	Checksum on load (T2, T5)

3.5 LINDDUN privacy threats in the real-world version

MNIST contains no personal data, so LINDDUN only becomes relevant for DigitRead, where every image is a crop of a real customer's handwriting.

Threat	How it arises in DigitRead	Mitigation
Linkability	Handwriting style links cheques to the same writer, even with names removed	Store only the digit crop; delete images after processing
Identifiability	Handwriting is a behavioural biometric; scans may include signatures, names or account numbers	Crop to the amount box before storage; mask other fields
Non-repudiation	Stored images plus predictions tie a person to each document longer than the bank needs	Retention limits; keep the decision, not the image
Detectability	Membership inference (ML04) can reveal that a person's cheques were in the training data	Differential privacy in training; labels-only output
Disclosure of information	A model that memorises samples can leak them through inversion (ML03); training stores hold raw scans	Encrypt and restrict training data; DP-SGD; no confidence scores exposed
Unawareness	Customers do not know their handwriting is kept to train models	Notice at account opening; an opt-out from training use
Non-compliance	India's Digital Personal Data Protection Act, 2023 requires notice, consent, purpose limitation and erasure on request; erasing a person from a trained model is hard	Documented purpose, retention schedule, retraining that can exclude erased records

STRIDE and LINDDUN overlap at ML03 and ML04. A model that memorises its training data is a security problem for its owner and, at the same time, a privacy problem for the people whose handwriting it learned from.

4. SAST findings and fixes

The three scanners found 8 issues in version 1 of our code. Three of them would have let a downloaded file run code through Python's pickle format. We fixed all 8 in version 2, and a final scan of the whole repository came back clean: no Bandit findings, no Semgrep

findings and no known vulnerable packages. We also wrote a test for each fix, because a scanner going quiet does not prove that a problem is gone. The full write-up is in `docs/SAST.md`, and the raw scanner output is in `scans/`.

4.1 Tools and process

Tool	Version	What it checks	Source
Bandit	1.9.4	Python security anti-patterns	OWASP Source Code Analysis Tools list
Semgrep	1.182.1, rule packs `p/python` and `p/trailofbits` (192 rules)	Pattern rules, including Trail of Bits' rules for pickle use in PyTorch and NumPy	OWASP Source Code Analysis Tools list
pip-audit	2.10.1	Known vulnerabilities in installed packages	Added for dependency checking, as OWASP's Component Analysis guidance recommends; not itself on an OWASP list

We committed each stage separately, so the code before and after the fixes can be compared on GitHub:

Stage	Commit	Evidence
Version 1, code as first written	`2c96180`	
Scans of version 1 ("before")	`4b24ea2`	`scans/before/`
Version 2: fixes, security tests, scans ("after")	`03d8c31`	`scans/after/`
Whole repository, including attack and defense code, scanned again	`51dfd4d`	`scans/final/`

Tool	Version 1 (6 files)	Version 2 (9 files)	Final (20 files)
Bandit	4 (2 Medium, 2 Low)	0	0
Semgrep	4, all marked blocking	0	0

Tool	Version 1 (6 files)	Version 2 (9 files)	Final (20 files)
pip-audit	6 advisories in 2 packages; torch not auditable	0; torch checked separately: 0	0; torch: 0

4.2 Findings and fixes

#	Finding	Found by	Version 1 location	Fix	Proven by
F1	Model file can run code when loaded (CWE-502)	Bandit B614; Semgrep pickles-in-pytorch	`src/evaluate.py:29` `torch.load(path, weights_only=False)`; `src/train.py:110` `torch.save(model, …)`	Save weights only plus a SHA-256 file; verify the checksum, then load with `weights_only=True` into the known architecture	Test runs a payload through the v1 loader, then shows the fixed loader refusing the same file; tamper and missing-checksum tests
F2	`weights_only=True` can be bypassed on PyTorch before 2.10 (CVE-2025-32434, CVE-2026-24747)	Dependency review with pip-audit's advisory data; invisible to both code scanners	The F1 fix itself	Loader refuses to run on PyTorch older than 2.10; the project uses 2.14.1	Version-guard tests (5 cases)
F3	Dataset loaded with pickle enabled (CWE-502)	Semgrep pickles-in-numpy (Bandit has no rule)	`src/evaluate.py:149` `np.load(…, allow_pickle=True)`	Checksum on save; on load verify it, `allow_pickle=False`, check shape, type, pixel range and labels	Pickled-object dataset refused; 4 malformed datasets rejected
F4	Download accepts any URL scheme and never checks the content	Bandit B310	`src/data.py:35` `urlretrieve(mirror + name, path)`	HTTPS-only client, SHA-256 pinned per file, 30 s timeout, 20 MB limit, atomic write	Non-HTTPS refused; no handler for `file://`; tampered MNIST file rejected
F5	`assert` used for input checks; `python -O` deletes them (CWE-703)	Bandit B101 (2)	`src/data.py:47`, `:55`	Explicit checks that raise an error	Code compiled as `python -O` compiles it: the v1 check lets a bad header through, the fixed checks still reject it (4 tests)
F6	DataLoader memory pinning (performance, not security)	Semgrep automatic-memory-pinning	`src/train.py:41`	Pin memory only when a GPU is present	Triaged

#	Finding	Found by	Version 1 location	Fix	Proven by
F7	Vulnerable dependencies: `filelock` 3.32.3 (3 CVEs), `setuptools` 78.1.0 (3 advisories)	pip-audit	Pulled in by torch from the PyTorch CPU index	Minimum safe versions in `requirements.txt`, installed from PyPI (now 4.2.4 and 84.2.1)	pip-audit after: 0
F8	PyTorch not auditable under its local name `2.14.1+cpu`	pip-audit skip message	Environment	Audit the release number `torch==2.14.1` directly	0 known vulnerabilities

The fixes changed how the model is saved and loaded, but not what it learns. When we retrained with version 2 we got exactly the same results as version 1 (accuracy 0.9909, loss 0.0297, macro F1 0.9908), and even the confusion matrix file was byte-for-byte identical. All 22 security tests pass.

4.3 What the scanners missed

Running the scanners also showed us where they fall short. Bandit missed the NumPy pickle problem (F3) and the `torch.save` of a whole model object (F1). Semgrep's Trail of Bits rules caught both, so using more than one tool paid off. Semgrep's rules, however, look at the shape of the code rather than what it does. Our first F1 fix was already safe but stayed flagged until we passed the load directly into `load_state_dict`. The same rule also accepts `weights_only=True` on PyTorch versions where that option is known to be broken (F2).

Two gaps were easy to miss. Inside a git repository, Semgrep's built-in ignore list silently skipped our `tests/` folder, so the first "after" scan covered 7 files instead of 9. We only noticed because we checked the file count, and a project `.semgrepignore` fixed it. Bandit, for its part, stopped reporting F4 as soon as `urlretrieve` was gone, even though our first replacement still accepted `file://` URLs. Only a test caught that.

Two deliberate exceptions remain, both in the tests, where two lines recreate the version 1 attack to prove that it was real. They are marked with `nosec` / `nosemgrep` comments that explain why. A Semgrep run with suppressions turned off reports exactly those two lines and nothing else.

5. Attack code and adversarial / poisoned datasets

We picked two attacks from the OWASP ML Top 10 and implemented both with IBM's Adversarial Robustness Toolbox (ART) 1.20.1. The first, evasion (ML01), attacks the model at prediction time through its inputs. The second, backdoor poisoning (ML02), attacks it at training time through its data.

Item	Link
Attack code (folder)	`attacks/`
Attack 1: evasion with FGSM and PGD	`attacks/evasion.py`
Attack 2: backdoor poisoning	`attacks/poison.py`
Stronger attacks used to check the defense (Section 7)	`attacks/robustness_checks.py`
Adversarial and poisoned datasets (folder)	`datasets/`

5.1 Attack 1: evasion with adversarial examples (OWASP ML01, Input Manipulation)

This attack exercises ML01, Input Manipulation. An attacker who controls an input image changes its pixels slightly, often by an amount a person would not notice, and the model predicts the wrong digit. For DigitRead, that would mean a cheque amount being read as a different number.

We chose it because every input to a deployed classifier is under the attacker's control (boundary TB4). That makes it the most direct threat to the system, and it needs no access to the training process. We ran it white-box, with full knowledge of the weights. This is the worst case for the defender, so it gives an upper bound on the damage and the hardest possible test for our defense.

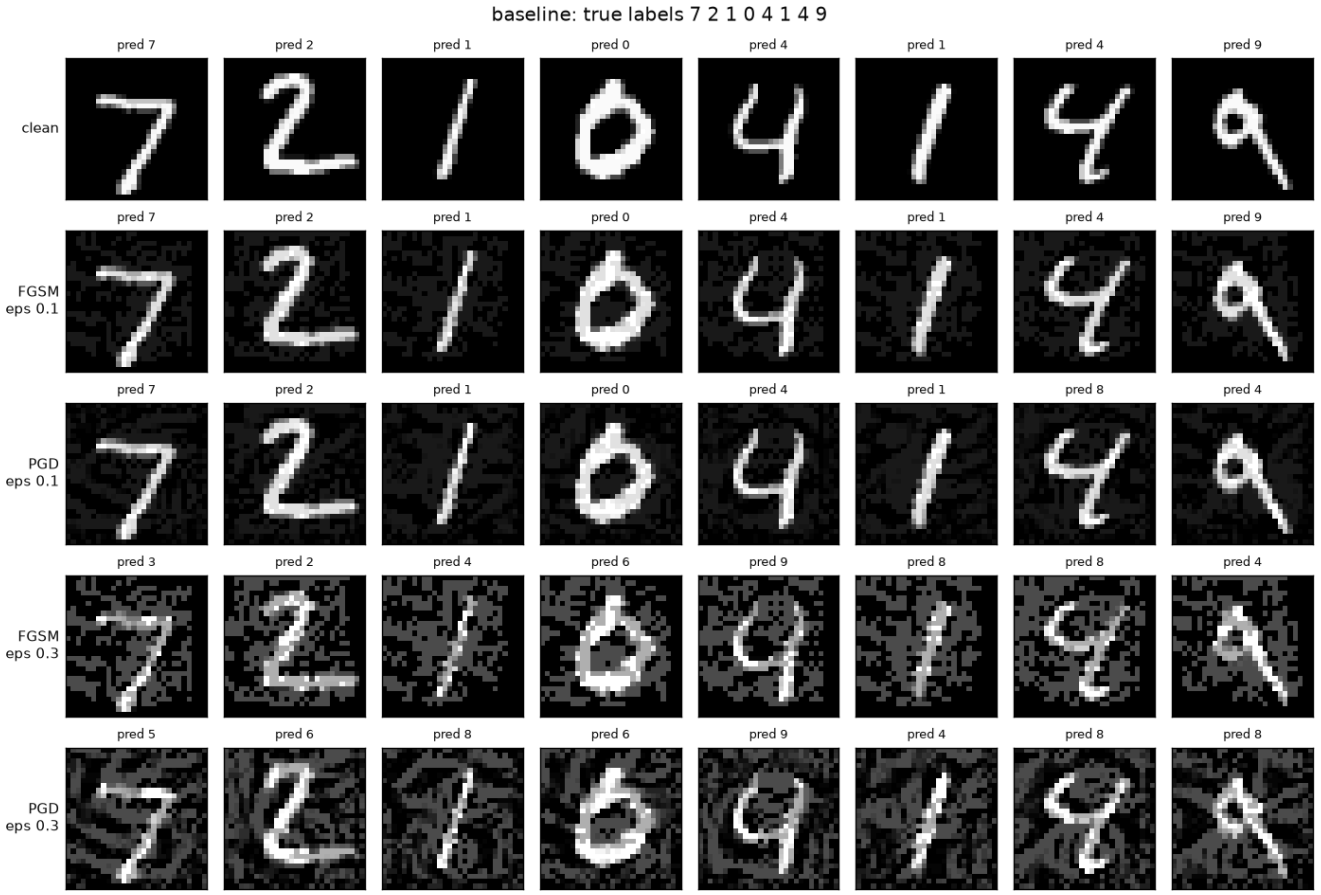
We used two methods. FGSM (Fast Gradient Sign Method) takes a single step of size ε in the direction that increases the loss:

$$x_{\text{adv}} = \text{clip}_{[0,\,1]}(x + \varepsilon \cdot \text{sign}(\nabla_x L(f(x),\ y)))$$

PGD (Projected Gradient Descent) is the stronger, iterative version. It starts from a random point within ε of the image and takes 20 smaller steps, projecting back into the allowed range after each one:

$$x^{t+1} = \Pi_{\|x' - x\|_\infty \le \varepsilon}(x^t + \alpha \cdot \text{sign}(\nabla_x L(f(x^t),\ y))), \quad \alpha = 2.5\,\varepsilon/20$$

Both attacks are untargeted, so any wrong digit counts as a success. Both are bounded in the L∞ norm, meaning no pixel may change by more than ε on the 0–1 scale. They use the true labels and run on all 10,000 test images at ε = 0, 0.05, 0.10, 0.15, 0.20, 0.25 and 0.30. We stopped at 0.3 because that is the standard benchmark strength for MNIST (Madry et al., 2018). When we report attack success rate, we count only the images the model got right before the attack.



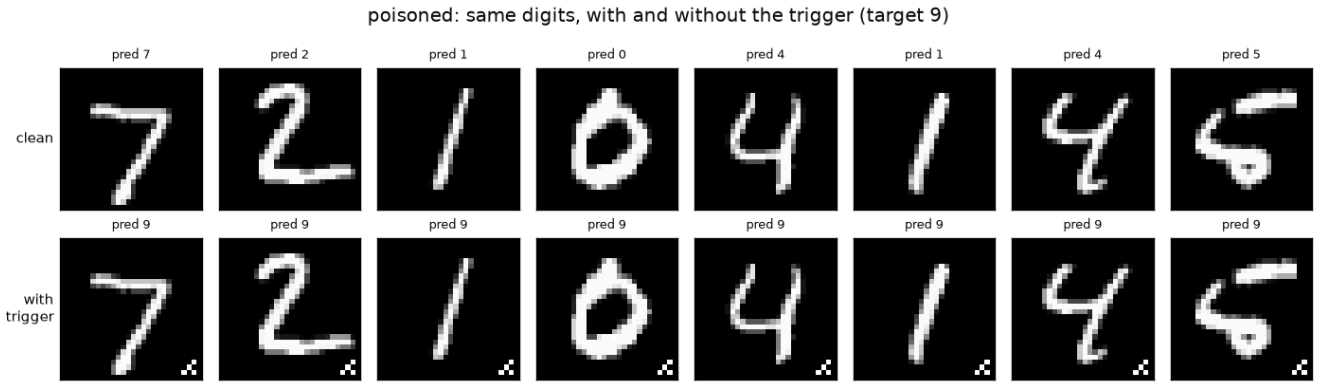
At ε = 0.1 the changes look like faint background noise, yet PGD already flips two of these eight digits. At ε = 0.3 PGD gets all eight wrong, although a person can still read every one of them easily.

5.2 Attack 2: backdoor data poisoning (OWASP ML02, Data Poisoning)

This attack exercises ML02, Data Poisoning. Someone who supplies part of the training data hides a rule inside the model. In our case the rule is that any digit carrying a small trigger pattern is read as a 9.

We chose it because it targets the other half of the system, training (boundary TB2), and needs no access to the model or the code. It also fits a realistic supply chain, since DigitRead relies on an outside labelling contractor. Backdoors are stealthy: clean accuracy stays the same, so normal testing does not reveal them. Finally, it needs a completely different defense from evasion, which gave the blue team two separate problems to solve.

We built a BadNets-style backdoor using ART's `PoisoningAttackBackdoor` with `add_pattern_bd`, which turns a small 4-pixel pattern near the bottom-right corner white. We randomly picked 2,750 training images (5% of 55,000) from digits other than 9, added the trigger and relabelled them as 9. The victim model was then trained with exactly the same code, data split and seed as the baseline, so the poison is the only difference between the two.



Without the trigger, the poisoned model reads all eight digits correctly. With it, every one of them becomes a 9.

5.3 Datasets

Each dataset is saved as a NumPy `.npz` file with a `.sha256` checksum file next to it. They should be loaded with `src.data.load_dataset(path)`, which checks the checksum, refuses pickled objects, and validates shapes, value ranges and labels.

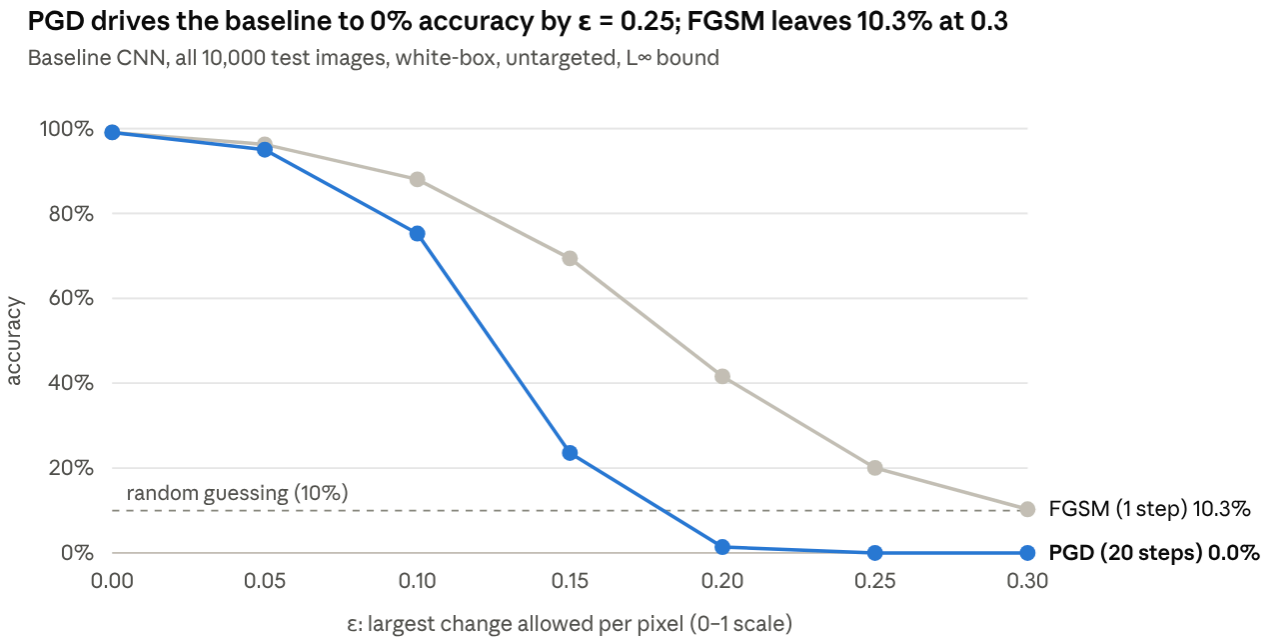
File	Contents	Size	SHA-256 (first 16 characters)
`baseline_fgsm_eps0.10.npz`	10,000 test images after FGSM, ε = 0.1, against the baseline; true labels	4.2 MB	`1cec3c2ce74a7808`
`baseline_fgsm_eps0.30.npz`	The same, ε = 0.3	3.9 MB	`70e01badf2302ada`
`baseline_pgd_eps0.10.npz`	10,000 test images after 20-step PGD, ε = 0.1	9.9 MB	`688b30b9a0a1c7dd`
`baseline_pgd_eps0.30.npz`	The same, ε = 0.3	10.5 MB	`4459db4944c6c531`
`poisoned_train.npz`	55,000 training images, 2,750 of them triggered and relabelled 9; also stores which images were poisoned and their original labels	14.5 MB	`778216425f3c4f1a`

File	Contents	Size	SHA-256 (first 16 characters)
`poisoned_triggered_test.npz`	The 8,991 test images that are not 9, each with the trigger; true labels	2.4 MB	`54d5da730a89df7c`

6. Performance metrics with adversarial and poisoned data

Under attack the baseline falls apart. PGD at ε = 0.3 takes its accuracy from 99.09% to 0.00%, and FGSM brings it down to 10.29%. The poisoned model keeps 99.10% accuracy on clean digits, but sends 99.94% of triggered digits to 9.

6.1 Adversarial test data (Attack 1)



results/baseline/evasion.json · 7 attack strengths × 2 attacks, 10,000 test images each

The table below gives the full metrics for the four adversarial test sets we saved. Confusion matrices and per-class reports for each are in `results/baseline_*`.

Test set (10,000 images)	Accuracy	Loss	Macro F1	Attack success	Weakest digit (recall)
Clean, for reference	99.09%	0.030	99.08%	—	8 (97.84%)
FGSM, ε = 0.1	88.02%	0.386	87.88%	11.17%	9 (77.60%)
FGSM, ε = 0.3	10.29%	3.98	10.65%	89.62%	1 (0.09%)
PGD, ε = 0.1	75.27%	0.813	74.97%	24.04%	9 (49.26%)

Test set (10,000 images)	Accuracy	Loss	Macro F1	Attack success	Weakest digit (recall)
PGD, ε = 0.3	0.00%	15.6	0.00%	100.00%	every digit (0%)

PGD is stronger than FGSM at every ε, which is what we expected: twenty projected steps find mistakes that a single step misses. This also served as a sanity check on our implementation. The loss tells a similar story. It rises from 0.03 to 15.6, which means the model is not merely uncertain on these images but confidently wrong.

We also checked whether the attack needs our weights at all. The PGD images made against the baseline fooled two other models that we trained separately with the same architecture. The poisoned model scored 0.00% on them and the cleaned model 1.38% (Section 7). In other words, an attacker with a similar model of their own could produce working adversarial examples without ever seeing ours.

6.2 Poisoned training data (Attack 2)

Model and test set	Images	Accuracy (true labels)	Loss	Read as 9
Baseline, clean test set	10,000	99.09%	0.030	—
Poisoned model, clean test set	10,000	99.10%	0.028	—
Poisoned model, triggered test set (no 9s)	8,991	0.06%	13.98	**99.94%**

What makes the backdoor dangerous is how well it hides. On clean data the poisoned model matches the baseline to within 0.01 points, and even its weakest digit is the same (8, at 97.84% recall), so an accuracy check before release would pass it. The backdoor works for every source digit, from 99.69% for digit 4 up to 100%. To make sure the effect came from the poison and not from the pattern itself, we also stamped the trigger onto test images for the clean baseline. Only 0.08% of them turned into a 9.

7. Blue team: defenses and performance compared with the baseline

Both defenses worked, and neither cost much. PGD adversarial training raised accuracy under PGD at ε = 0.3 from 0.00% to 90.34% (83.30% against the strongest attack we tried) and lowered clean accuracy by 0.55 points. Activation clustering cut the backdoor's success rate from 99.94% to 0.18%, for a drop of 0.18 points in clean accuracy.

One result surprised us. Training once on FGSM images, which is the most literal reading of the task, produced a model that looks robust against the baseline's own attack images. When attacked directly, however, it does worse than the undefended baseline.

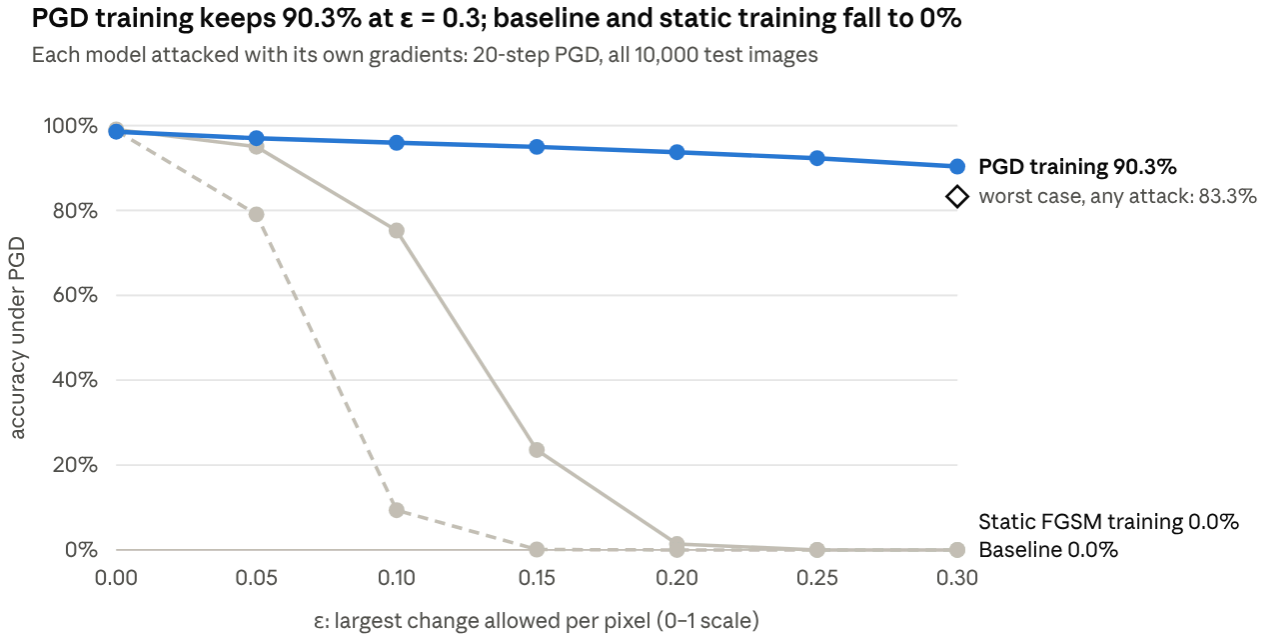
Item	Link
Defense 1: adversarial training (static and PGD)	`defenses/adversarial_training.py`
Defense 2: activation clustering, then retraining	`defenses/activation_clustering.py`
Comparison table and charts	`scripts/compare.py`, `results/comparison.md`
Complete output of the blue-team run	`results/blue_team.log`

7.1 Defense 1: adversarial training (against ML01)

The task asks us to train on adversarial samples together with clean data. We tried two ways of doing this, and the difference between them turned out to be one of our main findings:

	Static (adversarial data added once)	PGD adversarial training (Madry et al., 2018)
Adversarial samples	FGSM, ε = 0.3, made once against the baseline for all 55,000 training images	Fresh 7-step PGD, ε = 0.3, made against the model as it trains, for half of every batch
Training data	110,000 images (clean + adversarial), 5 epochs	55,000 images, half adversarial per batch, 8 epochs
Training time (CPU)	164 s	572 s (baseline: 74.5 s)

In both cases the adversarial samples were made from the training images, not taken from the adversarial test sets in Section 5, so that no test image was ever seen during training and the retest stays fair. For the PGD version, we generated the training attacks with our own PGD code but evaluated the model with ART's implementation, so the defense was never judged by the same code that trained it. Its accuracy on validation images under PGD went up after every epoch, from 54.4% after the first to 90.1% after the eighth.



results/{baseline,adv_static,adv_pgd}/evasion.json and results/adv_pgd/robustness_checks.json

The static model learned to recognise the particular FGSM perturbations it had seen. On the baseline's PGD images at ε = 0.3 it scores 90.05%, against 0.00% for the baseline. But when we attacked it with its own gradients, it broke down faster than the baseline did, falling to 9.37% under PGD at ε = 0.1 where the baseline still had 75.27%. Its training images were made for a different model, so it seems to have learned to spot one fixed pattern instead of becoming robust. The PGD-trained model behaved very differently: it kept 95.93% at ε = 0.1, 90.34% at ε = 0.3, and 94.02% under FGSM at ε = 0.3.

7.2 Is that robustness real? Checks for gradient masking

A defense can look robust simply because it hides or distorts its gradients. Gradient-based attacks then fail even though the model is still easy to fool, which is known as gradient masking. To check for this, we ran three tests suggested by Carlini et al. (2019), "On Evaluating Adversarial Robustness", on the first 1,000 test images:

Attack on the PGD-trained model	Accuracy	Check
None (clean)	98.20%	
PGD, 20 steps, 1 start, ε = 0.3 (main evaluation)	88.90%	
PGD, 100 steps, 5 random starts, ε = 0.3	86.70%	Pass: a 5× stronger attack gains only 2.2 points
Square Attack (black-box, no gradients), ε = 0.3	**83.30%**	**Fail:** beats every gradient attack

Attack on the PGD-trained model	Accuracy	Check
PGD, 50 steps, ε = 0.5	0.00%	Pass: a large enough budget breaks it completely

The black-box Square Attack, which never uses gradients, did better than every gradient attack. That points to some gradient masking. The model is still far more robust than the baseline, but on these images its true robust accuracy is at most 83.30%, not the 88.90% that PGD alone suggests. We therefore report 83.30% as its robust accuracy.

7.3 Defense 2: activation clustering (against ML02)

Activation clustering (Chen et al., 2018; ART's `ActivationDefence`) relies on a simple observation. Poisoned images reach their label because of the trigger rather than the shape of the digit, so inside the network they look different from genuine images of the same class. For each class, we took the poisoned model's activations on its training images, reduced them to 10 dimensions with PCA and split them into two clusters with k-means. A cluster holding less than 35% of its class was treated as poison, and we retrained the model on everything that was not flagged. ART uses the output layer by default, so we compared it with the hidden layer, each with three different rules for deciding which cluster is the poison:

Layer	Rule for the poison cluster	Images flagged	Precision	Recall
Hidden (128 units)	**Relative size (under 35% of the class)**	**2,624**	**99.9%**	**95.4%**
Hidden	The smaller cluster in every class	23,739	11.1%	95.4%
Hidden	Silhouette score	8,219	33.5%	100%
Output (ART default)	Relative size	5,951	39.4%	85.3%
Output	The smaller cluster in every class	22,111	10.6%	85.3%
Output	Silhouette score	13,661	20.1%	100%

With the setting we chose, 2,624 images were flagged, all of them in class 9. Of these, 2,622 really were poisoned and only 2 were clean, while 128 of the 2,750 poisoned images slipped through. After retraining on the remaining 52,376 images, the model scores 98.91% on clean data. The backdoor now works on only 0.18% of triggered digits (at most 0.71%

for any single source digit), so the 128 missed images were not enough to bring it back. The choice of layer made a big difference: on ART's default output layer, more than half of the flagged images were actually clean.

7.4 All models compared with the baseline

Model	Clean accuracy	FGSM ε = 0.1	FGSM ε = 0.3	PGD ε = 0.3	Baseline's PGD ε = 0.3 images	Backdoor success	ms per image	Training time
Baseline	99.09%	88.02%	10.29%	0.00%	0.00%	0.08% (control)	0.229	74.5 s
Poisoned (5%)	99.10%	—	—	—	0.00%	99.94%	0.233	73 s
Poison removed (activation clustering)	98.91%	—	—	—	1.38%	**0.18%**	0.226	68 s + detection
Adversarial training, static FGSM	98.80%	57.03%	3.87%	0.00%	90.05%	—	0.226	164 s
Adversarial training, PGD	98.54%	**96.56%**	**94.02%**	**90.34%** (83.30% worst case)	**95.19%**	—	0.225	572 s

In the FGSM and PGD columns each model is attacked with its own gradients (white-box). The "baseline's images" column instead replays the adversarial test set made against the baseline, which is a black-box transfer test. Inference time is the same for every model within measurement noise, because they all share one architecture; the defenses cost training time, not prediction time. A dash means we did not measure that value for the model.

8. Conclusion and observations

Our main takeaway is that high accuracy says very little about security. The baseline scored 99.09% on clean data and 0.00% under a white-box attack, and the backdoored model passed the same accuracy check as the clean one. Both attacks can be defended against without slowing down prediction, but only if the defense itself is tested with adversarial methods.

Observations

1. Accuracy did not reveal either attack. Evasion needed no access to training, only pixel changes of at most 0.3 that are barely visible. Poisoning 5% of the training data planted a backdoor that worked 99.94% of the time, while clean accuracy even went up by 0.01 points. Robustness and data provenance have to be tested in their own right.
2. The most obvious defense can make things worse. Adding FGSM images made once against the baseline gave a model that resists those exact images (90.05%) but is weaker than the undefended baseline under a direct attack (9.37% against 75.27% at ε = 0.1). Real robustness only came from PGD adversarial training, which keeps attacking the model while it learns and reached 90.34% under PGD at ε = 0.3.
3. A single attack is not enough to evaluate a defense. The black-box Square Attack beat every gradient attack against the PGD-trained model (83.30% against 86.70% for 100-step PGD), so we report 83.30% as its robust accuracy. Quoting only the PGD result would have overstated it by 5.6 points on the same images.
4. Default settings deserve a second look. Activation clustering on the hidden layer found the poison with 99.9% precision, but on ART's default output layer 60% of what it flagged was clean data. ART 1.20.1's own relative-size rule also failed with an error under NumPy 2, so we applied the rule ourselves to ART's clusters.
5. The defenses cost training time rather than prediction time. Every model predicts in 0.225–0.233 ms per image. Clean accuracy dropped by 0.55 points with PGD training and by 0.18 points with poison removal, and PGD training took 572 s compared with 74.5 s for the baseline.
6. The most serious code issues were specific to machine learning. Three of the eight SAST findings were ways for a model or dataset file to execute code through pickle, and it was the Trail of Bits ML rules, not Bandit, that caught them. Each scanner also had a blind spot: Semgrep skipped the tests folder and ignores the PyTorch version, and Bandit went quiet on a download fix that still accepted `file://`. That is why we treat the tests, not the scans, as the evidence that a fix works.
7. Dependencies need their own audit. Installing PyTorch from its CPU index pulled in outdated versions of `filelock` and `setuptools` with six known advisories between them. PyTorch itself could only be audited once we checked its release number directly.

Limitations

- We only tested L∞ perturbations up to ε = 0.3. Adversarial training gives no guarantee beyond the attacks we tried; that would need a certified defense.

- Activation clustering is designed for BadNets-style backdoors that form their own cluster. Clean-label attacks, or triggers spread across the whole image, might get past it.
- Each configuration was trained once, with seed 42, on a single CPU. Differences of a few tenths of a point, such as 99.10% against 99.09%, are within normal run-to-run variation.
- The checksum files are stored next to the files they protect. They catch corruption and casual tampering, but not an attacker who can replace both; signing the files would close that gap.

If we were deploying DigitRead for real, we would use the PGD-trained model, run activation clustering on every batch of contractor data before retraining, and keep a record of where each training sample came from. We would also sign the model files, and add differential privacy, rate limiting and label-only responses to deal with the privacy threats from Section 3.5.